Design Note: Resilient Multi-Agent Squad for NYC TLC Monthly Revenue

Spyne Member of Technical Staff Take-Home | Source: NYC TLC Yellow Taxi Trip Records (2024 vs. 2025)

1. Revenue Definition, Explicit Assumptions & Financial Defense

Under the NYC TLC Yellow Taxi Data Dictionary (March 18, 2025), `total_amount` is the total passenger charge (excluding cash tips). Raw monthly Parquet files exhibit three artifacts that corrupt naive `SUM(total_amount)` queries: (1) TPEP vendor dispute/void offsetting rows with negative charges (`total_amount <= 0` or `fare_amount < 0` from Curb Mobility `VendorID=2`), (2) taximeter clock-drift spillover records outside the target calendar month (`2023-12-31` or `2024-12-31` inside January files), and (3) 2025 schema/domain shifts (addition of `cbd_congestion_fee = $0.75/trip` starting Jan 5, 2025, lowercase `airport_fee`, and `payment_type = 0` *Flex Fare* trips where `congestion_surcharge` and `airport_fee` are `NULL`).

- **Primary Recognized Revenue (`gross_passenger_revenue_usd`)**: `ROUND(SUM(total_amount), 2)` over strictly in-month valid billable rides (`tpep_pickup_datetime >= YYYY-MM-01 AND tpep_pickup_datetime < next_month-01 AND total_amount > 0 AND fare_amount >= 0`).
- **Horizontal Reconciliation & Segregated Reversals**: Each `(pickup_zone, payment_type)` group reports `fare_revenue_usd`, `surcharges_and_fees_usd` (`COALESCE(extra,0) + COALESCE(mta_tax,0) + COALESCE(improvement_surcharge,0) + COALESCE(congestion_surcharge,0) + COALESCE(airport_fee,0) [+ COALESCE(cbd_congestion_fee,0)]`), `tip_revenue_usd`, and `tolls_revenue_usd`, which must sum to `gross_passenger_revenue_usd` within `$0.05`. Negative dispute reversals are isolated in `reversal_adjustments_usd` and netted in `net_total_amount_usd` so reversals never distort unit economics.
- **One-Line Assumptions**: (1) Revenue accrual cutoffs follow pickup timestamps (`tpep_pickup_datetime`). (2) Unmapped/NULL `PULocationID` rows join via `LEFT JOIN` to `LocationID = 264 (Unknown)` so zero revenue is dropped.

2. Agent Boundaries: Why the Split Earns Its Complexity (30%)

A monolithic agent that profiles data, writes SQL, and self-checks its output suffers from shared-context confirmation bias: if its SQL omits `COALESCE` on a nullable fee column, its self-check repeats the same bug. Our squad splits work across **three specialized agents and a stateful orchestrator** with asymmetric permissions and hard state firewalls:

Component	Module	Real Tools & Permissions	Boundary & Distinct Responsibility
Orchestrator	`squad/orchestrator.py`	`DurableStateStore` (SQLite ACID)	Enforces `max_steps=12`, `max_retries=3`, `max_cost_usd=$0.25`; manages crash-safe stage resumption; purges unverified outputs on refusal.
Agent 1: Profiler & Drift	`squad/profiler_agent.py`	`inspect_parquet_schema`, profiling SQL	Profiles Arrow types, null ratios, domain codes, temporal bounds, and row accounting vs. `schema_baseline_2024.json`; emits `DriftAssessment`.
Agent 2: SQL Builder	`squad/builder_agent.py`	`execute_readonly_sql`, CSV/JSON writer	Synthesizes aggregation SQL from `AdaptationPlan`. On repair retries, receives **only** Verifier rejection codes and remediation hints.
Agent 3: Verifier	`squad/verifier_agent.py`	`VerifierInput` firewall, 8 SQL audits	Sees **only** candidate CSV/JSON output, `revenue_contract.json`, and read-only raw tables. **Never** sees Builder reasoning, SQL, or drift notes.

3. Verifier Independence & Bounded Repair Loop (25%)

- **Hard Input Firewall (`VerifierIsolationError`)**: `VerifierInput` sets `extra = "forbid"` and `IndependentVerifierAgent.verify()` blocks `FORBIDDEN_BUILDER_KEYS` (`builder_reasoning`, `sql_query`,

`drift_assessment`, `applied_fixes`). Leaking any Builder state raises `VerifierIsolationError` (verified in `test_3_verifier_architectural_isolation_firewall`).

- **Eight Independent Raw-Data SQL Audits (`V1–V8`)**: The Verifier runs its own SQL queries against raw tables: `V1_SCHEMA_AND_COMPLETENESS` (18 columns, 0 NULL/NaN cells), `V2_TEMPORAL_BOUNDARY_AUDIT` (exact in-month trip count match, catching clock-drift spillover), `V3_GLOBAL_REVENUE_RECONCILIATION` (global gross, itemized sum, and net revenue match), `V4_ACCOUNTING_IDENTITY_CHECK` (row-level `|fare + surcharges + tip + tolls - gross| <= $0.05`), `V5_ZONE_JOIN_INTEGRITY` (zones in `1..265`), `V6_PAYMENT_TYPE_DOMAIN_CHECK` (codes in `{0..6}`), `V7_CELL_SPOT_CHECK` (5-cell point-by-point SQL audit), and `V8_STATISTICAL_SANITY_BOUNDS` (`$2.50 <= avg_rev_per_trip <= $500.00`).
- **Live Repair Loop on `2025-01`**: In `2025-01`, `payment_type = 0` (*Flex Fare*) introduces `NULL` values in `congestion_surcharge` and `airport_fee`. On **Attempt 1**, the Builder maps the new 2025 columns (`cbd_congestion_fee`, `airport_fee`) but sums fees without row-level `COALESCE`, causing SQL `NULL` propagation to zero out `surcharges_and_fees_usd` on Flex Fare cells (`components_sum = $10,947.20` vs. `gross = $12,018.45` in Zone 161). The Verifier catches this via `V4_ACCOUNTING_IDENTITY_CHECK` (`REJECT_V4_ACCOUNTING_IDENTITY_MISMATCH`) and rejects Attempt 1. On **Attempt 2**, the Builder wraps each fee column in `COALESCE(col, 0.0)` and passes all `8/8` Verifier checks (`$2,689,193.04` verified gross revenue).

4. Drift Handling vs. Human Escalation & Refusal (25% Drift / 20% Judgement)

Drift Tier	Profiler Trigger Conditions	Squad Action	Committed Trace Evidence
1. Clean (`CLEAN`)	Matches 19-column 2024 baseline; payment types in `{1..6}`; <5% row accounting mismatch.	`PROCEED_CLEAN`	`trace_1_clean_pass_2024-01`: 0 retries, 8/8 checks passed, `$2,354,271.15` gross revenue.
2. Adaptable (`ADAPTABLE_DRIFT`)	Case drift (`Airport_fee` → `airport_fee`), statutory fee addition (`cbd_congestion_fee`), extended enum (`payment_type=0` Flex Fare), nullable fee columns.	`ADAPT_AND_PROCEED` (+ Repair Loop)	`trace_2_drift_and_repair_2025-01`: 5 drift findings adapted; Attempt 1 rejected (`V4`); Attempt 2 accepted (`$2,689,193.04`).
3. Fatal Refusal (`FATAL_REFUSAL`)	Missing required column; unrecognized monetary column; `>0.5%` unmapped `payment_type`; `>5%` rows failing accounting reconciliation; `<80%` in-month rows; or retry exhaustion (`max_retries=3`).	`STOP_AND_REFUSE` (Purge outputs & escalate)	`trace_3_refusal_2025-99`: Caught `14.08%` unmapped `payment_type=99` and `38.01%` corrupted `total_amount` rows (+$25 offset). Refused in 1 step.

5. Honesty About Where It Breaks

- **Semantic Label Swap Within Valid Codes**: If a vendor firmware bug swaps `payment_type = 1` (Credit Card) and `payment_type = 2` (Cash) while zeroing credit card tips so horizontal accounting (`V4`) still holds, single-month verification passes. Catching this requires cross-month historical distribution priors (alerting if credit card share drops from ~75% to <25%).
- **Coordinated Sub-Threshold Underbilling**: If both `fare_amount` and `total_amount` are simultaneously understated by the exact same delta on every trip (preserving `fare + surcharges + tip + tolls == total_amount` and staying within `[$2.50, $500.00]` per trip), SQL self-consistency cannot detect it without external bank settlement feeds.
- **In-Memory SQLite Ceiling Without DuckDB**: When `duckdb` is absent, `SQLDataEngine` stages Parquet tables into in-memory SQLite via PyArrow, which handles single monthly files (50k–3.5M rows) on standard hosts but requires out-of-core DuckDB streaming for multi-year (50M+ row) backfills on low-memory containers.